FLUTTER MINI PROJECT

Mini Project Topic:

College Placement Tracker

TaskFlow is a mobile task and project manager built with Flutter and a SQLite database. A user can add tasks with a validated form, see them in a searchable list, move them across a Kanban board, view deadlines on a calendar and check progress on an analytics page. Every task has a title, description, project, priority, status and due date, and the data is saved on the device so it stays after the app is closed. The project shows how Flutter's core widgets and database operations (create, read, update and delete) work together.

Team Members

G.Tejaswari-24501A4416

K.Sushma-24501A4428

P.Srilatha-24501A4445

N.Tejaswi-24501A4441

GitHub: https://github.com/gugulothuteja29-bit/flutterpro

Deployment: https://flutterpro-yznnucmsb6yji7fjjuqmeq.streamlit.app/

---

1. Aim

To design and build a mobile application in Flutter that manages structured data with a SQLite database. The application must have an intuitive user interface made with Flutter's core widgets (Form, TextFormField, DropdownButton, ListView and others) and must show how to **create, read and display, update, and remove** data using database operations.

2. Description

TaskFlow is a task and project manager. Each task has a title, description, project, priority, status and due date. Tasks are stored in a SQLite database on the phone, so they remain after the app is closed. The user can add a task with a validated form, see all tasks in a searchable list, edit a task or mark it complete, and delete it after a confirmation. The same tasks are shown on five pages: Dashboard, Task List, Kanban Board, Calendar and Analytics, opened from the side menu.

Main features

- **Dashboard:** totals for all, completed, in-progress and urgent tasks, overall completion bar and a list of tasks needing attention.
- **Task List:** search box and filters by category, status and priority; edit, delete and tick-to-complete on every task.
- **Kanban Board:** To Do, In Progress and Completed lanes with buttons that move a task between lanes.
- **Calendar:** monthly grid showing each task on its due date.
- **Analytics:** efficiency score, workload and breakdowns by status and priority.
- **Validation and messages:** required fields are checked before saving; SnackBars report success or errors.

Technologies and widgets

Item	Use in the project
Flutter and Dart	Framework and language for the user interface.
sqflite and path	SQLite database on the phone and the database file path.
Form, TextFormField, DropdownButtonFormField	New Task / Edit Task form with validation; drop-downs for project, priority and status.

Item	Use in the project
ListView.builder, Card, ListTile	Task list and cards.
Scaffold, AppBar, Drawer	Page structure and the side menu with five pages.
AlertDialog, SnackBar, showDatePicker	Form dialog, delete confirmation, messages and due-date selection.
VS Code and Android Studio	Code editing, Android SDK and emulator.

Database table `tasks`

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
title	TEXT	Not null
description	TEXT	Optional
project, priority, status	TEXT	Not null (project name; Urgent / High / Medium / Low; To Do / In Progress / Completed)
dueDate	TEXT	Not null, stored as yyyy-MM-dd

3. Workflow

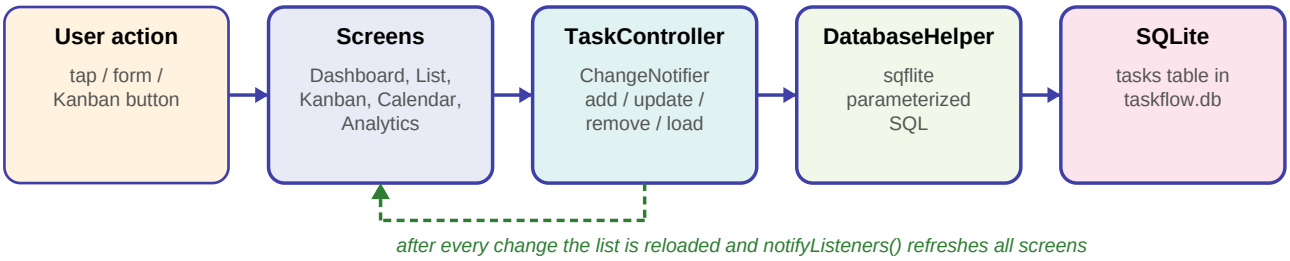
3.1 How data moves through the app

The screens never talk to SQLite directly. A **TaskController** sits between the screens and the **DatabaseHelper**, so all database code is in one place and every screen shows the same data.

3.2 Step-by-step workflow of the application

1. **Start:** `main()` runs `TaskFlowApp`. Its home page, `HomeShell`, creates the TaskController and calls `load()`.
2. **Read:** `load()` calls `DatabaseHelper.getTasks()`. On the first run the database file and the `tasks` table are created and four sample tasks are added.
3. **Display:** a ListenableBuilder rebuilds the screen whenever the controller changes. The Drawer switches between the five pages, which all read `ctl.tasks`.
4. **Create:** the New Task button opens the form dialog. The form checks the title, project and due date; if valid, the new Task is passed to `ctl.add()`, which inserts a row.
5. **Update:** the pencil icon reopens the form with the task's values. The tick and the Kanban buttons change only the status through `ctl.setStatus()`, which runs an UPDATE using the task id.
6. **Delete:** the bin icon shows an AlertDialog; only after the user confirms does `ctl.remove()` run a DELETE using the id.
7. **Refresh and message:** after each change the controller reloads the list and calls `notifyListeners()`, so counts, board, calendar and charts update together. A SnackBar shows success or an error.

3.3 Data flow diagram



3.4 Development workflow

Step	Work done
1	Create the Flutter project and add the `sqflite` and `path` packages in `pubspec.yaml`.
2	Create the Task model with `toMap` and `fromMap` (models/task.dart).
3	Write DatabaseHelper: open the database, create the table, insert, read, update and delete (database/database_helper.dart).
4	Write TaskController to hold the task list and refresh the screens (controllers/task_controller.dart).

Step	Work done
5	Build the form and dialogs (widgets/task_dialogs.dart) and the five screens (screens/).
6	Connect everything in `main.dart` with the Drawer, the New Task button and the page switch.
7	Run on an emulator or phone with `flutter run`, test add, edit, delete and validation, then push the code to GitHub.

4. Complete main.dart Code

File: `lib/main.dart`. It starts the app, sets the theme, and defines `HomeShell`, which holds the TaskController, the AppBar with the New Task button, the Drawer menu and the switch between the five screens. It imports the controller, database helper, model, screens and dialog helpers, which are in their own files.

```
import 'package:flutter/material.dart';
import 'controllers/task_controller.dart';
import 'database/database_helper.dart';
import 'models/task.dart';
import 'screens/analytics_screen.dart';
import 'screens/calendar_screen.dart';
import 'screens/dashboard_screen.dart';
import 'screens/kanban_screen.dart';
import 'screens/task_list_screen.dart';
import 'widgets/task_dialogs.dart';
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const TaskFlowApp());
}
class TaskFlowApp extends StatelessWidget {
  const TaskFlowApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'TaskFlow',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF4F46E5)),
      ),
      home: const HomeShell(),
    );
  }
}
class HomeShell extends StatefulWidget {
  const HomeShell({super.key});
  @override
  State<HomeShell> createState() => _HomeShellState();
}
class _HomeShellState extends State<HomeShell> {
  static const List<String> _titles = <String>[
    'Dashboard', 'Task List', 'Kanban Board', 'Calendar', 'Analytics',
  ];
  static const List<IconData> _icons = <IconData>[
    Icons.dashboard_outlined,
    Icons.checklist,
    Icons.view_kanban_outlined,
    Icons.calendar_month_outlined,
    Icons.bar_chart,
  ];
  final TaskController _ctl = TaskController(DatabaseHelper.instance);
  int _index = 0;
  @override
  void initState() {
    super.initState();
    _ctl.load();
  }
  @override
  void dispose() {
    _ctl.dispose();
    super.dispose();
  }
  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: _ctl,
      builder: (context, _) => Scaffold(
        appBar: AppBar(
          title: Text(_titles[_index]),
          actions: <Widget>[
            IconButton(
              tooltip: 'Notifications',
              icon: const Icon(Icons.notifications_none),
              onPressed: () => ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('No new notifications')),
              ),
            ),
            Padding(
              padding: const EdgeInsets.only(right: 8),
```

```
              child: FilledButton.icon(
                onPressed: () => addTask(context, _ctl),
                icon: const Icon(Icons.add),
                label: const Text('New Task'),
              ),
            ),
          ],
        ),
        drawer: _drawer(context),
        body: _body(),
      ),
    );
  }
  Widget _drawer(BuildContext context) {
    return Drawer(
      child: SafeArea(
        child: ListView(
          children: <Widget>[
            const ListTile(
              title: Text('TaskFlow',
                  style: TextStyle(fontWeight: FontWeight.bold, fontSize: 20)),
              subtitle: Text('Project Suite'),
            ),
            const Divider(),
            for (var i = 0; i < _titles.length; i++)
              ListTile(
                leading: Icon(_icons[i]),
                title: Text('${i + 1}. ${_titles[i]}'),
                selected: _index == i,
                trailing: i == 1 ? Text('${_ctl.tasks.length}') : null,
                onTap: () {
                  setState(() => _index = i);
                  Navigator.of(context).pop();
                },
              ),
            const Divider(),
            const Padding(
              padding: EdgeInsets.fromLTRB(16, 8, 16, 4),
              child: Text('PROJECTS & TEAMS',
                  style: TextStyle(fontWeight: FontWeight.bold, fontSize: 12)),
            ),
            for (final e in projectColors.entries)
              ListTile(
                dense: true,
                leading: Icon(Icons.circle, size: 12, color: e.value),
                title: Text(e.key),
                trailing:
                    Text('${_ctl.tasks.where((t) => t.project == e.key).length}'),
              ),
          ],
        ),
      ),
    );
  }
  Widget _body() {
    if (_ctl.loading) {
      return const Center(child: CircularProgressIndicator());
    }
    if (_ctl.error != null && _ctl.tasks.isEmpty) {
      return Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: <Widget>[
            Text(_ctl.error!),
            const SizedBox(height: 12),
            FilledButton(onPressed: _ctl.load, child: const Text('Retry')),
          ],
        ),
      );
    }
    return switch (_index) {
      0 => DashboardScreen(ctl: _ctl),
      1 => TaskListScreen(ctl: _ctl),
      2 => KanbanScreen(ctl: _ctl),
      3 => CalendarScreen(ctl: _ctl),
      _ => AnalyticsScreen(ctl: _ctl),
    };
  }
}
```

5. Screenshots of All Pages Displayed in Chrome

The five pages of the application, captured in the Chrome browser at `http://localhost:5173/`.

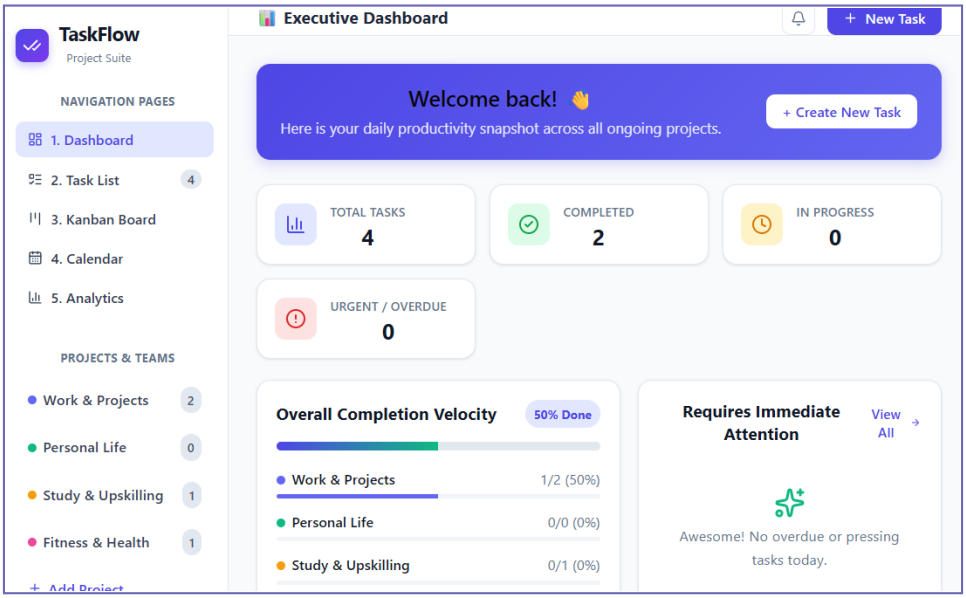


Fig. 5.1 Dashboard - Executive Dashboard

Shows Total Tasks (4), Completed (2), In Progress (0) and Urgent / Overdue (0), the overall completion bar at 50% with progress for each project, and the Requires Immediate Attention panel. The side menu lists the five pages and the four projects with their task counts.

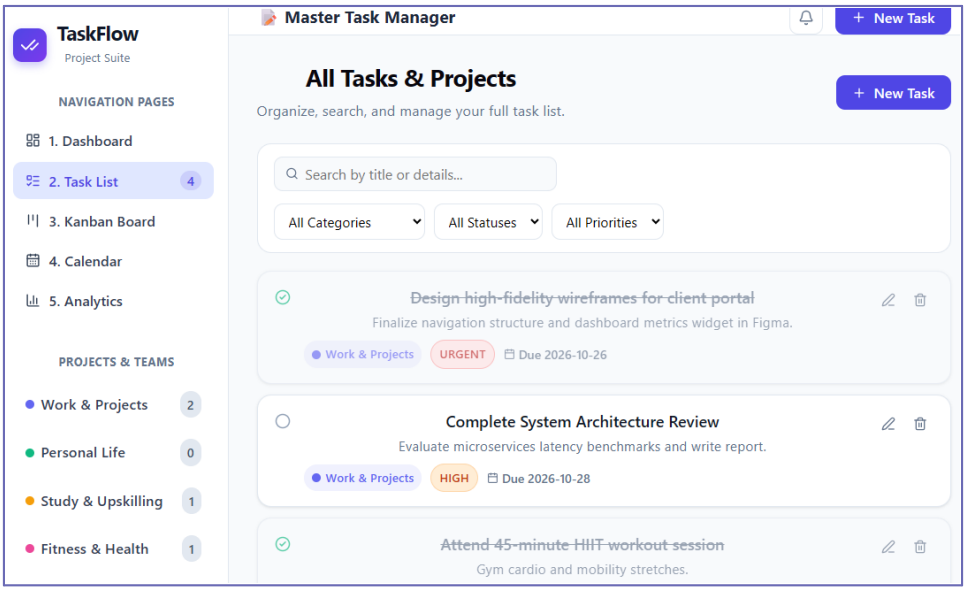


Fig. 5.2 Task List - Master Task Manager

All tasks with a search box and three filters (category, status, priority). Each card shows the project, a priority label (URGENT, HIGH) and the due date. Completed tasks are struck through; the pencil edits and the bin deletes.

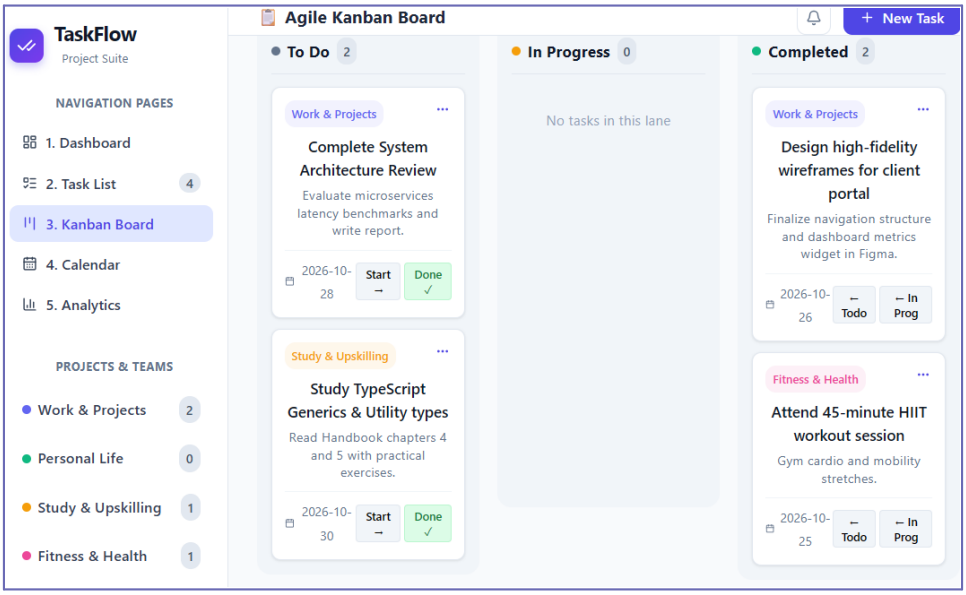


Fig. 5.3 Kanban Board - Agile Kanban Board

The same tasks in three lanes: To Do (2), In Progress (0) and Completed (2). The Start, Done and back buttons move a card to another lane and update its status in the database.

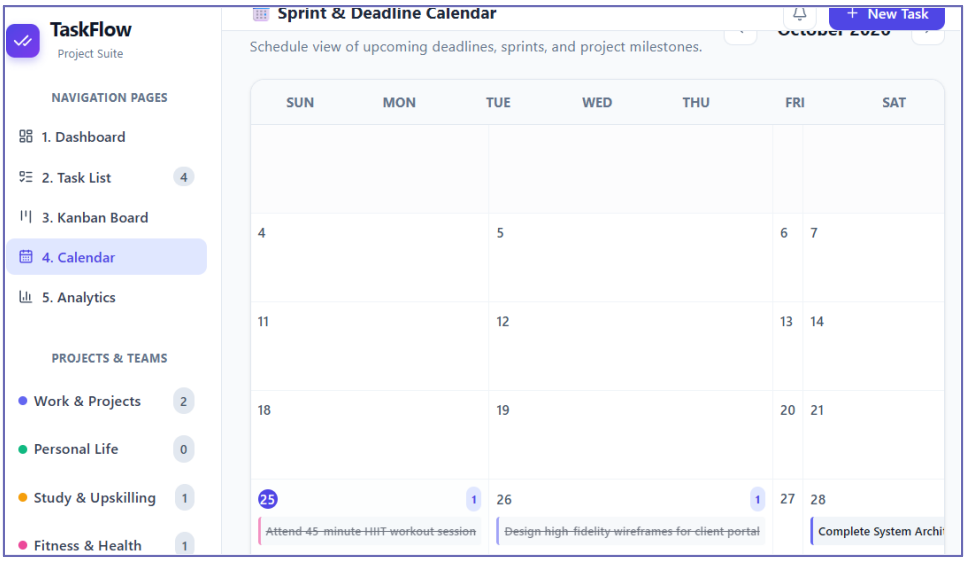


Fig. 5.4 Calendar - Sprint & Deadline Calendar (October 2026)

A monthly grid, Sunday to Saturday, with tasks placed on their due dates in the project colour. Today (25) is highlighted and a badge shows the number of tasks on a day. This screenshot shows the upper part of the month.

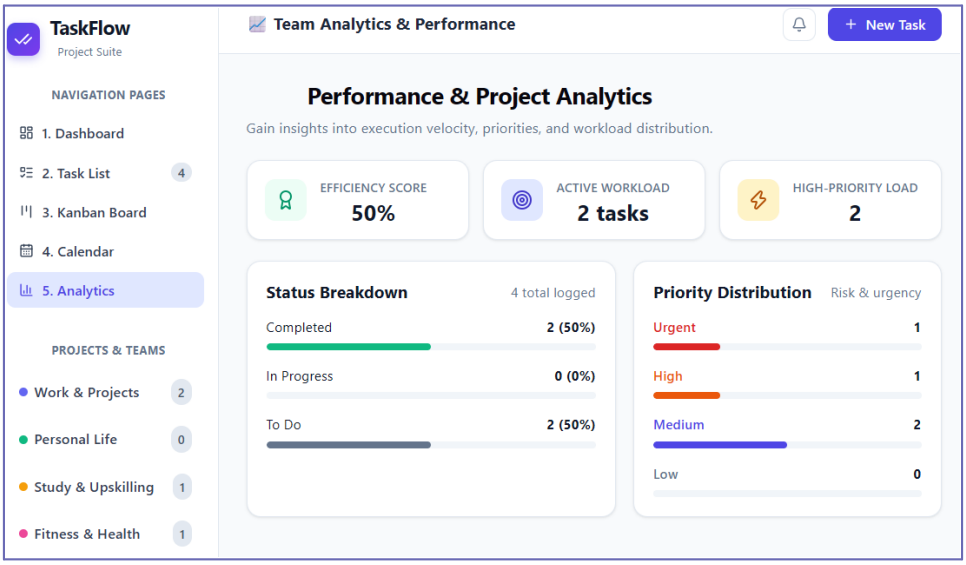


Fig. 5.5 Analytics - Performance & Project Analytics

Efficiency Score 50%, Active Workload 2 tasks and High-Priority Load 2, with Status Breakdown and Priority Distribution bars.

6. Conclusion

The TaskFlow mini project achieved its aim: a Flutter application with a clean user interface that stores and manages structured data in a SQLite database. Flutter's core widgets worked together to build the interface: Form and TextFormField collect and validate input, DropdownButtonFormField offers fixed choices, ListView.builder with Card and ListTile displays the tasks, and AlertDialog and SnackBar confirm deletions and report results.

All four database operations were demonstrated. Tasks are **created** through the form, **read and displayed** on five pages (Dashboard, Task List, Kanban Board, Calendar and Analytics), **updated** through the edit form, the tick and the Kanban buttons, and **deleted** after confirmation. Keeping the database code in DatabaseHelper and the shared task list in TaskController meant every page always showed the same, up-to-date data, and the data remained after the app was closed.

The project gave practical experience with widgets, form validation, asynchronous database calls, state updates and organising code into separate files. In future the application can be extended with user login, task reminders and notifications, and cloud synchronisation so tasks are available on more than one device.